

Mini Space Invaders

Documentazione del modello ASP e dei risultati sperimentali

Mattia Lupinetti M. 303595

Sommario

Questo documento descrive la codifica in Answer Set Programming (ASP) del problema *Mini Space Invaders*, secondo la specifica che richiede di trovare, tramite un CP/ASP solver, un piano che impedisca a qualsiasi astronave aliena di toccare Terra.

Indice

1	Model	2
1.1	Definizione del Dominio	2
1.2	Analisi dello Stato e Identificazione dei Bersagli	2
1.3	Generazione delle Azioni)	2
1.4	Transizione di Stato e Inerzia	3
1.5	Dinamica degli alieni	3
1.6	Condizione di Vittoria e Vincoli	3
1.7	Ottimizzazione delle Performance	3
1.8	Output	4
2	Tests	4
2.1	Le istanze di test	4
2.2	Analisi dei risultati	5
2.3	Visualizzazione piani	6
3	Conclusione.	7
A	Semantica degli atomi del modello ASP	8

1 Model

Il modello, riportato integralmente nel file `mini_space_invaders_model_new.lp`, segue il classico schema *Generate and Test* arricchito con direttive di ottimizzazione. In questa sezione ogni sotto-sezione corrisponde a un *cluster logico* del programma e ne illustra il ruolo e la corrispondenza con la specifica. Per agevolare la comprensione del modello la semantica di ogni atomo è riportata nella Tabella 2 dell'appendice A.

1.1 Definizione del Dominio

```
1 time(0..n).
2 xcoord(X) :- maxx(M), X = 0..M.
```

Queste due regole definiscono l'universo in cui si svolge il problema: un timeout discreto da 0 a n (con n costante fornita in input) e le coordinate X valide per il cannone e per le astronavi, comprese tra 0 e maxx . Viene fornito in input anche $\text{maxy}(n)$ con n costante discreta usata per effettuare un controllo di integrità sulle posizioni (iniziali) degli alieni con $Y < \text{MaxY}$:

```
1 alien(IDAlien,X,Y-1,T+1) :- time(T), alien(IDAlien,X,Y,T), maxy(MaxY), Y < MaxY, not
    shot(IDAlien,X,T), Y-1 >= 0.
```

1.2 Analisi dello Stato e Identificazione dei Bersagli

```
1 dangerous_alien(IDAlien,X,MinY,T) :-
2     alien(IDAlien,X,MinY,T),
3     MinY = #min { Y : alien(_,X,Y,T) }.
```

Per ogni colonna X e in ogni istante T la regola individua, tramite l'aggregato `#min`, l'alieno con coordinata Y minore, cioè quello più vicino alla Terra. Solo tale alieno è un bersaglio legalmente accessibile. Tale meccanica di fuoco permette al cannone, nell'eventualità in cui l'istanza di input abbia due alieni dalla stessa ascissa, di colpire l'alieno con la ordinata minima.

1.3 Generazione delle Azioni

```
1 possible_action(move(NewX,T),T) :-
2     time(T), T+1 < n, cannon(X,T), xcoord(NewX), NewX != X.
3
4 possible_action(shoot(IDAlien,X,T),T) :-
5     time(T), T < n, cannon(X,T), dangerous_alien(IDAlien,X,_,T).
6
7 1 { do(A) : possible_action(A,T) } 1 :-
8     time(T), T < n, possible_action(_,T), not game_over(T).
```

Le prime due regole definiscono lo spazio delle azioni legali ad ogni istante T :

- spostare il cannone in qualsiasi ascissa NewX diversa da quella attuale (l'azione `move` occupa esattamente un istante di tempo, indipendentemente dalla distanza percorsa)
- sparare (`shoot`) solo se nella propria colonna X esiste un `dangerous_alien`.

La choice rule `1 { do(A) : ... } 1` impone al solver di selezionare esattamente un'azione per ogni istante in cui esiste almeno una azione possibile e la partita non è terminata (*not world_saved*). La dinamica del cannone ha sempre tempo unitario (il passaggio da T a $T+1$ avviene a prescindere da $|\text{NewX} - X|$) ad ogni spostamento *move*.

1.4 Transizione di Stato e Inerzia

```
1 move(NewX,T) :- do(move(NewX,T)).
2 shoot(IDAlien,X,T) :- do(shoot(IDAlien,X,T)).
3 shot(IDAlien,X,T) :- shoot(IDAlien,X,T).
4
5 cannon(NewX,T+1) :- move(NewX,T), T+1 < n.
6 cannon(X,T+1) :- cannon(X,T), time(T), T+1 < n,
7                     not do(move(_,T)).
```

Queste regole aggiornano la posizione del cannone: se si decide di muoversi, al tempo $T+1$ il cannone si trova in $NewX$; in caso contrario entra in gioco la *regola di inerzia* (`not do(move(_,T))`) e il cannone mantiene la coordinata X del tempo precedente. Il predicato `shot/3` registra gli alieni colpiti per l'azione di fuoco. Viene quindi garantita la coerenza spaziale del cannone durante l'avanzamento del tempo (requisito necessario per problemi di pianificazione).

1.5 Dinamica degli alieni

```
1 alien(IDAlien,X,Y-1,T+1) :- time(T), alien(IDAlien,X,Y,T), maxy(MaxY), Y <= MaxY, not
    shot(IDAlien,X,T), Y-1 >= 0.
```

Se un alieno non è stato colpito nel turno corrente, al turno successivo la sua ordinata Y scende di 1. La condizione $Y-1 \geq 0$ fa sì che un alieno che raggiunge l'ordinata 0 (Terra) cessi di avanzare e renda impossibile la vittoria (si veda 1.6).

1.6 Condizione di Vittoria e Vincoli

```
1 world_saved :-
2     alien(_,_,Y,_), Y > 0,
3     Aliens = #count { ID : alien(ID,_,_,_) },
4     Shot = #count { IDAlien : shot(IDAlien,_,_) },
5     Aliens == Shot.
6
7 :- not world_saved.
8
9 game_over(T) :- time(T), not alien(_,_,_,T).
```

`world_saved` è vero solo se il numero totale di alieni introdotti nel modello coincide esattamente con il numero di alieni che hanno subito un'azione `shot` e se nessun alieno ha $Y \leq 0$ (ovvero è arrivato sulla terra). Il vincolo di integrità `:- not world_saved.` elimina ogni answer set in cui la Terra non viene salvata. Il predicato `game_over/1` blocca la generazione di azioni inutili (mosse o spari) dopo l'eliminazione dell'ultimo alieno, riducendo lo spazio di ricerca.

1.7 Ottimizzazione delle Performance

```
1 finish(T) :- T = #max { T1 : shot(_,_,T1); 0 }.
2 #minimize { T@2 : finish(T) }.
3
4 distance(D,T) :- cannon(X1,T), cannon(X2,T+1), finish(F),
5                     T < F, D = |X2-X1|.
6 #minimize { D@1,T : distance(D,T) }.
```

Pur esistendo molteplici piani validi, queste direttive istruiscono il solver a preferire il piano “migliore” secondo una gerarchia di priorità:

- **Priorità 2 (@2):** `finish(T)` individua l'istante dell'ultimo sparo e lo si minimizza — *cerchiamo dei modelli che verifichino `world_saved` nel minor tempo possibile*;
- **Priorità 1 (@1):** a parità di tempo impiegato, si minimizza la distanza complessiva percorsa dal cannone ($D = |X_2 - X_1|$).

Fondamentale per evitare piani illogici (es. piani con spostamenti ridondanti inutili).

1.8 Output

```
1 #show cannon/2.
2 #show move/2.
3 #show shot/3.
4 #show alien/4.
```

Le direttive `#show` restringono l'output ai soli predicati rilevanti: stato degli alieni, mosse, spari e posizione del cannone, fornendo il piano leggibile.

2 Tests

I test sono effettuati nel notebook `test_space_invaders.ipynb` il quale è diviso in tre sezioni principali:

- Libraries import and constant
- Test
- Computed plans

Il notebook salva i risultati in `/results/multiple_instances_test.csv`, importa il modello `ASP mini_space_invaders_model.lp` e nel contesto della sezione `3. Computed Plans` utilizza la funzione `animate_model` per graficare i piani calcolati definita in `utils.py`.

2.1 Le istanze di test

Le istanze sono definite nel file `instances.py` come dizionario Python con chiavi `istanza_1` ... `istanza_10_unsat`. Ogni istanza descrive formalmente una configurazione iniziale del gioco e viene tradotta in fatti ASP passati al modello della Sezione 1. I campi di ciascuna istanza sono:

- `n` (intero) *Timeout*: numero massimo di istanti di tempo del modello; corrisponde alla costante `n` usata in `time(0..n)`.
- `dim_x` fatto `maxx(M)`. — dimensione orizzontale del rettangolo di gioco (ascissa massima).
- `dim_y` fatto `maxy(M)`. — dimensione verticale del rettangolo di gioco (ordinata massima).
- `pos_cannon` fatto `pos_cannon(X)`. — ascissa iniziale del cannone. Sempre 0 in tutte le istanze, in accordo con la specifica, tuttavia, si è scelto di fornire la possibilità di modificare la posizione iniziale per dare maggiori l'opportunità di sperimentare tali varianti senza modificare il modello.
- `cannon` regola `cannon(X,0) :- pos_cannon(X)`. — posiziona il cannone all'istante 0 nella colonna indicata.
- `aliens` lista di fatti `alien(ID, X, Y, 0)`. — insieme delle astronavi presenti, ciascuna con identificatore univoco `ID` e coordinate intere iniziali (X, Y) , tutte con $Y > 0$.

Le prime nove istanze sono *soddisfacibili* e seguono un pattern di difficoltà crescente: crescono la dimensione del campo di gioco (5×5 fino a 30×30), il numero di alieni (da 2 a 9) e il timeout `n`. Nell'istanza_9 compaiono, inoltre, due alieni `tpos` e `galactus` a ordinata 10 per testare la regola `dangerous_alien`. La decima istanza, `istanza_10_unsat`, è costruita per essere *insoddisfacibile*: 10 alieni allineati sulla medesima ordinata $Y=1$, che raggiungeranno tutti la Terra dopo un solo istante di tempo — un solo cannone, con un'azione per turno, non può distruggerli tutti.

2.2 Analisi dei risultati

I risultati sperimentali sono raccolti nel file `multiple_instances_test.csv`, il cui schema è il seguente:

`exp_id` identificativo dell'istanza;

`x_size`, `y_size` dimensioni del rettangolo di gioco;

`time_out` timeout `n`;

`initial_cannon_pos` ascissa iniziale del cannone;

`aliens` numero di astronavi presenti;

`solve_time` tempo complessivo di risoluzione (ricerca + ottimizzazione), in secondi;

`sat_time` tempo per determinare la soddisfacibilità, in secondi;

`models` numero di answer set esaminati dal solver;

`optimal_models` numero di answer set ottimi trovati.

La Tabella 1 riporta i dati per le dieci istanze.

Istanza	<i>x</i>	<i>y</i>	<i>n</i>	Alieni	<code>solve_time</code> (s)	<code>sat_time</code> (s)	Modelli	Ottimi
istanza_1	5	5	4	2	0.0000	0.0000	0	0
istanza_2	7	7	6	3	0.0018	0.0003	1	1
istanza_3	10	10	6	3	0.0035	0.0008	2	1
istanza_4	12	12	8	4	0.0116	0.0008	3	1
istanza_5	15	15	10	5	0.0340	0.0036	4	1
istanza_6	20	15	10	5	0.0442	0.0122	1	1
istanza_7	20	20	14	7	0.2137	0.0098	4	1
istanza_8	30	20	14	7	0.4586	0.1393	6	1
istanza_9	30	30	20	9	2.0328	0.0900	15	1
istanza_10_unsat	30	10	20	10	0.0000	0.0000	0	0

Tabella 1: Risultati sperimentali sulle dieci istanze. Le *colonne* descrivono, nell'ordine: l'identificativo dell'istanza (`exp_id`); le dimensioni del rettangolo di gioco, ascissa *x* (`x_size`) e ordinata *y* (`y_size`); il timeout *n* (`time_out`); il numero di astronavi presenti inizialmente (`aliens`); il tempo complessivo di risoluzione con ottimizzazione `solve_time` (s); il tempo per determinare la soddisfacibilità `sat_time` (s); il numero di answer set esaminati (`models`); e il numero di answer set ottimi trovati (`optimal_models`). L'ultima riga (`istanza_10_unsat`) è l'unica configurazione insoddisfacibile: nessun modello e tempo nullo.

Crescita della difficoltà. Il tempo di risoluzione cresce monotonicamente con la dimensione dell'input. Le istanze piccole (`istanza_1`–`istanza_4`) vengono risolte in pochi millisecondi; da `istanza_5` in poi il tempo supera i 30 ms, e le istanze maggiori (`istanza_7`–`istanza_9`) richiedono tra circa 0.21 e 2.03 secondi. La tendenza è chiaramente non lineare: raddoppiare il numero di alieni e il timeout (da 2 alieni/ $n=4$ a 9 alieni/ $n=20$) comporta un aumento di circa tre ordini di grandezza del tempo di risoluzione. Questo è coerente con la complessità della pianificazione: il numero di sequenze di azioni possibili cresce esponenzialmente con il timeout n , mentre la fase di ottimizzazione (con i due livelli di `minimize`) richiede di esplorare più answer set prima di certificare l'ottimalità. Il numero di modelli esaminati cresce infatti da 1–4 (istanze piccole) fino a 15 per `istanza_9`.

Interessante notare come il `sat_time` rimanga sempre molto contenuto e cresca molto più lentamente del `solve_time`: è la fase di ottimizzazione a dominare il costo computazionale, segno che la fase di “generate” del modello è ben vincolata, mentre il “test” con i due livelli di priorità richiede esplorazioni aggiuntive.

L'istanza insoddisfacibile. `istanza_10_unsat` evidenzia il corretto funzionamento del vincolo di integrità :- `not world_saved.`: con 10 alieni su ordinata $Y=1$ e un'unica azione per turno, nessun piano può distruggerli tutti prima che tocchino Terra. Il solver rileva l'insoddisfacibilità immediatamente (tempo 0.0 s), senza produrre alcun modello (`models = 0`, `optimal_models = 0`). Il caso conferma che il modello non “inventa” soluzioni inesistenti e che la codifica della sconfitta (alieno che raggiunge $Y=0$ e non può più essere conteggiato tra quelli colpiti) interagisce correttamente con il vincolo di vittoria.

2.3 Visualizzazione piani

Nella sezione `3.Computed Plans` del notebook è possibile visualizzare per ogni istanza di test il modello calcolato dal solver sia in un formato tabellare (nella sottosezione `3.1 Tabular representation`) e sia in un formato animato (nella sottosezione `3.2 Graphical visualization of the computed plans`) dove per ogni modello è possibile visualizzare le azioni pianificate dal solver ad ogni istante T come mostrato nelle immagini riportate nella figura 1.

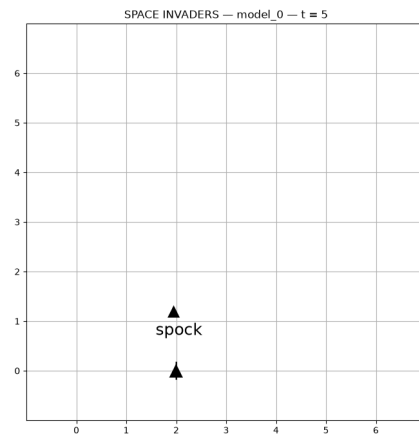
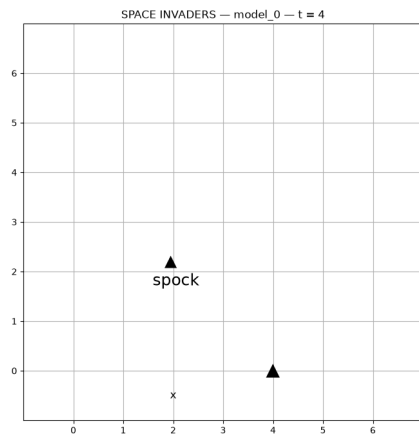
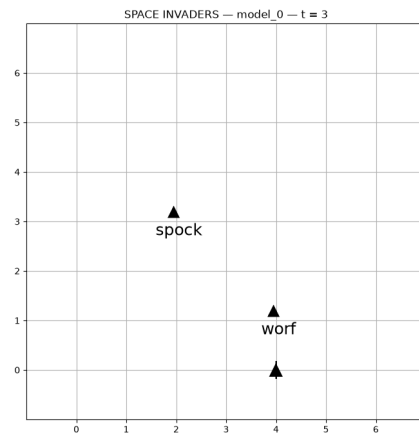
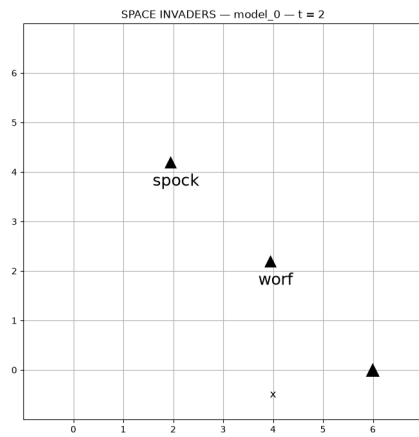
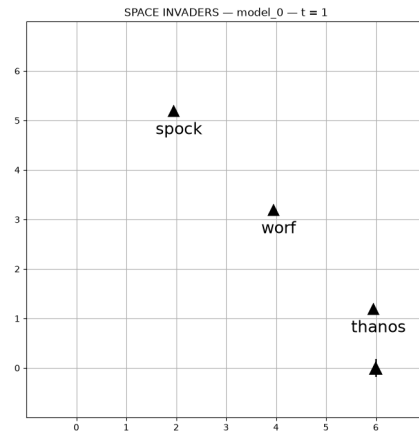
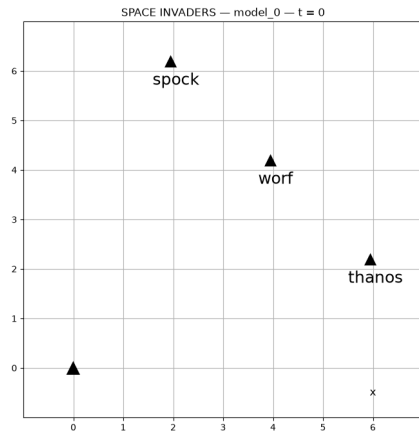


Figura 1: Sequenza temporale da $t = 0$ a $t = 5$ del modello calcolato per l'istanza di test 2 .

3 Conclusione.

Il modello ASP proposto codifica fedelmente tutti i requisiti della specifica, dalle meccaniche di movimento degli alieni e del cannone allo sparo, fino alla condizione di vittoria. I risultati

sperimentali mostrano tempi di poco superiori ai due secondi, e un comportamento corretto e immediato nel riconoscimento delle istanze insoddisfacibili.

A Semantica degli atomi del modello ASP

La Tabella 2 riporta, per ciascun atomo del modello ASP, la semantica dei suoi argomenti.

Atomo	Par.	Significato
maxx(M)	M	ascissa massima della board (larghezza)
maxy(M)	M	ordinata massima della board (altezza, quota massima di partenza degli alieni)
pos_cannon(X)	X	ascissa iniziale del cannone
time(T)	T	istante temporale discreto, $T \in \{0, \dots, n\}$
xcoord(X)	X	ascissa valida sulla board, $X \in \{0, \dots, \text{maxx}\}$
cannon(X,T)	X T	ascissa occupata dal cannone istante a cui il cannone si trova in X
alien(ID,X,Y,T)	ID X Y T	identificativo dell'alieno ascissa dell'alieno (costante nel tempo) ordinata dell'alieno (quota residua rispetto al suolo) istante a cui l'alieno si trova in (X, Y)
dangerous_alien (ID,X,MinY,T)	ID X $MinY$ T	alieno più vicino al suolo nella colonna X ascissa considerata quota minima (più pericolosa) tra gli alieni istante di riferimento
possible_action (move(NewX,T),T)	$NewX$ T	ascissa di destinazione della mossa istante in cui la mossa è eseguibile
possible_action (shoot(ID,X,T),T)	ID X T	alieno bersaglio ascissa di sparo istante in cui lo sparo è eseguibile
do(A)	A	azione scelta dal solver (<code>move(...)</code> o <code>shoot(...)</code>)
move(NewX,T)	$NewX$ T	nuova ascissa verso cui si sposta il cannone istante dello spostamento (il cannone sarà in $NewX$ a $T+1$)
shoot(ID,X,T)	ID X T	alieno colpito ascissa dello sparo istante in cui avviene lo sparo
shot(ID,X,T)	ID X T	alieno colpito ascissa a cui si trovava l'alieno colpito istante in cui è stato colpito
world_saved	—	(0-ario) vero se tutti gli alieni sono stati colpiti
game_over(T)	T	istante da cui non esiste più alcun alieno vivo
finish(T)	T	istante dell'ultimo sparo effettuato nel piano
distance(D,T)	D T	distanza percorsa dal cannone spostandosi tra T e $T+1$ istante di partenza dello spostamento

Tabella 2: Semantica degli argomenti degli atomi del modello ASP